

第一次课后作业

选择题

3.2

- (1) D
- (2) B
- (3) A
- (4) C
- (5) B
- (6) D
- (7) D

简答题

3.3 (4)

题目：

(4) 为什么阵列除法器中能用 CAS 的进位/借位控制端作为上商的控制信号？

答：

阵列除法器在执行除法运算时，通常利用的是不恢复余数除法（也称加减交替法）。

在由可控加减单元（CAS, Controlled Add/Subtract）构成的阵列中，每一行负责计算出一位商。判断当前商是上 1 还是上 0 的核心逻辑在于“是否够减”。

- **当商上 1 时：**这意味着被除数（或上一层的部分余数）够减去除数。此时运算结果为正，最左侧（最高位）的 CAS 单元会产生一个进位信号（通常表现为 $C_{out} = 1$ ）。
- **当商上 0 时：**这意味着部分余数不够减去除数。此时运算结果为负，最左侧的 CAS 单元不会产生进位信号（通常表现为 $C_{out} = 0$ ）。

综上所述可以发现，最高位 CAS 产生的进位信号的逻辑状态（1 或 0），与当前这一步运算应该上的商位状态（1 或 0）完全吻合。因此，在硬件电路的设计中，可以直接把最左侧 CAS 的进位/借位输出端引出，直接作为当前位的商输出，同时该信号也会向下传递，作为下一层 CAS 阵列是执行加法还是减法的控制信号。这种巧妙的设计省去了额外的判断逻辑，极大简化了运算器的电路结构。

3.3 (5)

题目：

(5) 移位运算和乘法及除法运算有何关系？

答：

移位运算是实现计算机底层乘法和除法运算的核心基础操作。在运算器硬件实现中：

- 乘法运算本质上是“移位”与“加法”操作的循环组合来实现的（即移位累加）。
- 除法运算本质上是“移位”与“减法”（或加减交替）操作的循环组合来实现的。

3.5 (3)

题目：

已知 x 和 y ，用变形补码（双符号位）计算 $x - y$ ，并判断结果是否溢出。

(3) $x = -0.11111, y = -0.11001$ 。

答：

首先分别求出 x 的变形补码和 $-y$ 的变形补码：

- 由 $x = -0.11111$ ，其原码为 11.11111 ，反码为 11.00000 ，加 1 得到补码：

$$[x]_{\text{补}} = 11.00001$$

- 由 $y = -0.11001$ ，可知 $-y = +0.11001$ ，正数的补码与原码相同：

$$[-y]_{\text{补}} = 00.11001$$

进行加法运算 $[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$ ：

$$\begin{array}{r} 11.00001 \\ +00.11001 \\ \hline 11.11010 \end{array}$$

计算结果为 $[x - y]_{\text{补}} = 11.11010$ 。

由于结果的两位符号位为 11，符号位相同，表明结果未溢出。

3.7 (2)

题目：

用补码一位乘法计算 $x \times y$ 。

(2) $x = -0.011010, y = -0.011101$ 。

答：

采用补码一位乘法（Booth算法），部分积和被乘数采用双符号位，右移规则为算术右移。

首先求出 x 和 y 的补码，以及 $-x$ 的补码：

由 $x = -0.011010$ ，得 $[x]_{\text{补}} = 11.100110$ ， $[-x]_{\text{补}} = 00.011010$

由 $y = -0.011101$ ，得 $[y]_{\text{补}} = 1.100011$

部分积 P 初值为 00.000000，乘数寄存器 Y 初值为 1100011，附加位 y_{n+1} 初值为 0。

步数	$Y_n Y_{n+1}$	操作	P (部分积)	Y (乘数)	y_{n+1}
0		初始状态	00.000000	1100011	0
1	10	$+[-x]_{\text{补}}$	00.011010		
		右移一位	00.001101	0110001	1
2	11	$+0$	00.001101		
		右移一位	00.000110	1011000	1
3	01	$+ [x]_{\text{补}}$	11.101100		
		右移一位	11.110110	0101100	0
4	00	$+0$	11.110110		
		右移一位	11.111011	0010110	0
5	00	$+0$	11.111011		
		右移一位	11.111101	1001011	0
6	10	$+ [-x]_{\text{补}}$	00.010111		
		右移一位	00.001011	1100101	1
7	11	$+0$	00.001011		
		不移位	00.001011	1100101	1

最终结果由部分积 P 和乘数寄存器 Y 的高 6 位组成，即双符号位下 $[x \times y]_{\text{补}} = 00.001011110010$ 。

转换为单符号位表示： $[x \times y]_{\text{补}} = 0.001011110010$ 。

3.8 (1)

题目：

用原码不恢复余数法计算 $x \div y$ 。

(1) $x = 0.10101, y = 0.11011$ 。

答：

被除数和除数均为正数，商的符号位单独计算： $Q_s = 0 \oplus 0 = 0$ 。

原码运算取绝对值参与计算，并采用双符号位表示：

$[|x|]_{补} = 00.10101$

$[|y|]_{补} = 00.11011$

$[-|y|]_{补} = 11.00101$

计算过程如下：

部分余数	商	操作说明
00.10101	0.00000	初始被除数
+11.00101		减去除数 $[- y]_{补}$
11.11010	0.00000	余数<0, 上商 $q_0 = 0$
11.10100		左移一位
+00.11011		加除数 $[y]_{补}$
00.01111	0.10000	余数>0, 上商 $q_1 = 1$
00.11110		左移一位
+11.00101		减去除数 $[- y]_{补}$
00.00011	0.11000	余数>0, 上商 $q_2 = 1$
00.00110		左移一位
+11.00101		减去除数 $[- y]_{补}$
11.01011	0.11000	余数<0, 上商 $q_3 = 0$
10.10110		左移一位 (注：此处产生溢出)
+00.11011		加除数 $[y]_{补}$
11.10001	0.11000	余数<0, 上商 $q_4 = 0$
11.00010		左移一位
+00.11011		加除数 $[y]_{补}$
11.11101	0.11000	最后一步余数<0, 上商 $q_5 = 0$, 需恢复余数
+00.11011		加除数 $[y]_{补}$
00.11000		最终恢复的余数

结合商的符号位，得商 $x \div y = 0.11000$ 。

最终真值余数需要乘以 2^{-5} ，即 $r = 0.11000 \times 2^{-5} = 0.11 \times 2^{-5}$ 。

3.10 (2)

题目：

采用 IEEE754 单精度浮点数格式计算下列表达式的值。

(2) $0.625 - (-12.25)$

答：

$0.625 = (0.101)_2 = (0\ 01111110\ 010000000000000000000000)_2$

$-12.25 = -(1100.01)_2 = (1\ 10000010\ 100010000000000000000000)_2$

(a) 对阶。

阶差 $[\Delta E]_{\text{补}} = [E_x]_{\text{补}} + [-E_y]_{\text{补}} = 01111110 - 10000010 = -00000100 = -4$ 。

将 0.625 的尾数右移 4 位，考虑隐藏位则尾数为 0.000101。

(b) 尾数计算 $0.000101 - (-1.100010) = 1.100111$ 。

(c) 结果规格化。

尾数符合 1.X 的形式，无须规格化。

(d) 舍入处理、溢出判断。

尾数无须舍入，阶码正常，无溢出，则

$0.625 - (-12.25) = (0\ 10000010\ 100111000000000000000000)_2 = (414E0000)_{16}$

3.11

题目：

3.11 假定在一个 8 位字长的计算机中运行如下 C 语言程序段。

```
unsigned int x=134;
unsigned int y=246;
int m=x; int n=y;
unsigned int z1=x-y;
unsigned int z2=x+y;
int k1=m-n;
int k2=m+n;
```

若编译器编译时将 8 个 8 位寄存器 R1~R8 分别分配给变量 x、y、m、n、z1、z2、k1 和 k2。请回答下列问题（提示：带符号整数用补码表示）。（可在 C 环境下给出运行结果，并结合结果进行分析）

(1) 执行上述程序段后，寄存器 R1、R5 和 R6 中的内容分别是什么？（用十六进制表示）

(2) 执行上述程序段后，变量 m 和 k1 的值分别是多少？（用十进制表示）

(3) 上述程序段涉及带符号整数加减、无符号整数加减运算，这 4 种运算能否利用同一个加法器及辅助电路实现？简述理由。

(4) 计算机内部如何判断带符号整数加减运算的结果是否发生溢出？上述程序段中，哪些带符号整数运算语句的执行结果会发生溢出？

答：

为了在现代编译器中严格模拟 8 位字长环境，测试代码采用 `<stdint.h>` 中的 8 位无符号和有符号整型。代码如下：

```
#include <stdio.h>
#include <stdint.h>
uint8_t x=134,y=246,z1,z2;
int8_t m,n,k1,k2;
int main(){
    m=x,n=y;
    z1=x-y,z2=x+y;
    k1=m-n,k2=m+n;
    printf("R1(x)=%02X, R5(z1)=%02X, R6(z2)=%02X\n",x,z1,z2);
    printf("m=%d, k1=%d\n",m,k1);
    return 0;
}
```

运行结果：

```
C 3.11.c  ×
C 3.11.c > main()
1  #include <stdio.h>
2  #include <stdint.h>
3  uint8_t x=134,y=246,z1,z2;
4  int8_t m,n,k1,k2;
5  int main(){
6      m=x,n=y;
7      z1=x-y,z2=x+y;
8      k1=m-n,k2=m+n;
9      printf("R1(x)=%02X, R5(z1)=%02X, R6(z2)=%02X\n",x,z1,z2);
10     printf("m=%d, k1=%d\n",m,k1);
11     return 0;
12 }
```

问题 输出 调试控制台 终端 端口 串行监视器 筛选器(例如 text、!excludeTe... Code

```
[Running] cd "c:\Users\vintces\OneDrive - stu.xmu.edu.cn\学校\课程\大二下\计组\第
11 && "c:\Users\vintces\OneDrive - stu.xmu.edu.cn\学校\课程\大二下\计组\第二次课后
R1(x)=86, R5(z1)=90, R6(z2)=7C
m=-122, k1=-112

[Done] exited with code=0 in 2.74 seconds
```

详细的计算与分析如下：

(1) 运行结果显示 `R1(x)=86, R5(z1)=90, R6(z2)=7C`。

已知分配的都是 8 位寄存器，无符号整数 x 和 y 在底层的机器码计算如下：

- $x = 134 = 128 + 6 = (1000\ 0110)_2$ ，转换为十六进制为 86H。因此 R1 中的内容为 86H，与运行结果相符。
- $y = 246 = 255 - 9 = (1111\ 0110)_2$ 。
- $z1 = x - y$ 。在计算机底层，减法通过加上减数的补码实现。 $[-y]_{\text{补}} = 0000\ 1010$ 。

计算过程： $1000\ 0110 + 0000\ 1010 = 1001\ 0000 = 90\text{H}$ 。因此 R5 中的内容为 90H，与运行结果相符。

- $z2 = x + y$ 。

计算过程： $1000\ 0110 + 1111\ 0110 = (1)\ 0111\ 1100 = 7\text{CH}$ （最高位产生的进位 1 被丢弃，只保留 8 位）。因此 R6 中的内容为 7CH，与运行结果相符。

(2) 运行结果显示 `m=-122, k1=-112`。

- m 是由 x 赋值而来的 8 位有符号整数。它的机器数与 x 完全相同，皆为 $86\text{H} = (1000\ 0110)_2$ 。最

高位为 1，表示负数。求其原码真值：取反加一得到 $(0111\ 1010)_2 = 64 + 32 + 16 + 8 + 2 = 122$ 。因此 m 的十进制值为 -122，与运行结果相符。

- $k1 = m - n$ 。底层机器数运算规律与 $x - y$ 完全相同，结果机器码同为 $90H = (1001\ 0000)_2$ 。解释为带符号整数时，最高位为 1，表示负数。求其原码真值：取反加一得到 $(0111\ 0000)_2 = 64 + 32 + 16 = 112$ 。因此 $k1$ 的十进制值为 -112，与运行结果相符。

(3) 能。

n 位加法器实现的是模 2^n 的加法运算。对于无符号整数的加法 $a + b$ 可以直接用加法器实现；对于减法 $a - b$ ，可用 a 加上 b 的补数实现，即 $a - b = a + [-b]_{\text{补}} \pmod{2^n}$ 。由于带符号整数采用补码表示，其符号位直接参与运算，加减法规则与无符号数在位级操作上完全一致。因此，这 4 种运算都可在同一个 n 位加法器及辅助电路（如求补电路）中实现。

(4) 判断溢出的规则：在计算机内部，如果加法器最高数据位的进位 C_d 与符号位的进位 C_f 不同（即 $C_f \oplus C_d = 1$ ），则带符号整数加减运算发生溢出。

上述程序段中，最后一条语句（`k2=m+n`）执行时会发生溢出。

计算分析： m 的机器数为 1000 0110， n 的机器数为 1111 0110。

相加过程：1000 0110 + 1111 0110 = (1) 0111 1100。

在这个加法过程中，符号位产生的进位 $C_f = 1$ ，而最高数据位产生的进位 $C_d = 0$ 。两者不同，因此发生溢出（两个负数相加得到了一个形似正数的结果）。